

# Manuale amministratore

Versione 0.2.0-11 · build 20261001-10:16:44

Repository: <https://github.com/desalvo/mta-audio-editor> · EUPL-1.2

## Indice

1. Architettura
2. Docker
3. Kubernetes
4. Configurazione
5. Demucs/stem plugin
6. Pipeline audio
7. Sicurezza
8. CI/CD
9. Operazioni
10. Backup
11. Release
12. Responsabilità production
13. Plugin custom e Auto Mix

# 1. Architettura

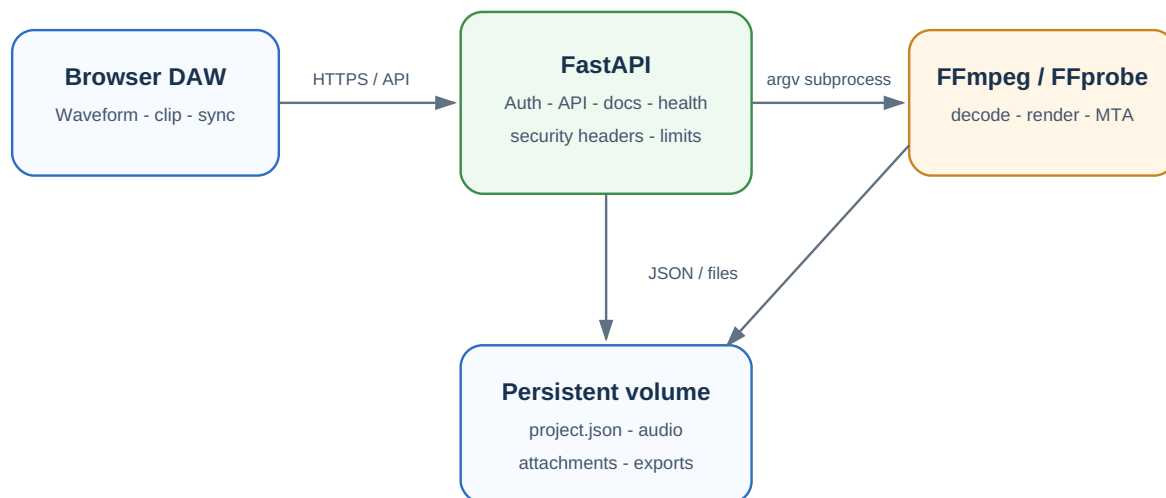


Figura 1 - Browser, FastAPI, FFmpeg, plugin audio e storage persistente.

| Componente      | Responsabilità                                                  |
|-----------------|-----------------------------------------------------------------|
| FastAPI/Uvicorn | UI, API, autenticazione, health e documentazione.               |
| FFmpeg/FFprobe  | Probe, rendering clip, insert, summing, export e container MTA. |
| Demucs plugin   | Separazione MP3 in stems indipendenti.                          |
| /data/projects  | Project JSON, sorgenti, attachment, model cache ed export.      |

## 2. Docker

```
export MTA_ADMIN_PASSWORD='password-lunga-casuale'  
docker compose up -d --build
```

L'immagine production include FFmpeg e, per default, il plugin Demucs. Il container gira come UID/GID 10001, senza capability e con no-new-privileges. La root filesystem è read-only nel deployment consigliato.

Per una build ridotta senza stem splitter:

```
docker build --build-arg INSTALL_STEMS=false -t mta-audio-editor:core
```

### 3. Kubernetes / Kustomize

---

**Obbligatorio:** proteggere il Secret. `secret.example.yaml` non è incluso nel Kustomization di base; in produzione preferire External Secrets, Sealed Secrets, Vault o equivalente.

Le risorse sono separate in Namespace, PVC, Deployment e Service sotto `k8s/base`. Gli esempi Ingress sono disponibili in `k8s/overlays/nginx` e `k8s/overlays/haproxy`.

```
kubectl apply -k k8s/overlays/nginx  
# oppure  
kubectl apply -k k8s/overlays/haproxy
```

Il wizard standalone `scripts/k8s-wizard.py` genera manifest locali completi chiedendo username/password amministrativi, StorageClass, namespace e nodeSelector. Ricorda gli ultimi valori in un file locale `0600`, può produrre un Ingress NGINX/HAProxy e controlla automaticamente una versione più recente su GitHub; se si aggiorna sostituisce atomicamente il proprio file e si riavvia da solo.

```
curl -fsSL mta-k8s-wizard.py https://raw.githubusercontent.com/desal  
python3 mta-k8s-wizard.py
```

Il PVC montato su `/data/projects` contiene anche la cache Demucs/Torch, così i modelli non devono essere riscaricati a ogni restart.

## 4. Configurazione

| Variabile                  | Default                     | Descrizione               |
|----------------------------|-----------------------------|---------------------------|
| MTA_ADMIN_USERNAME         | admin                       | Utente HTTP Basic.        |
| MTA_ADMIN_PASSWORD         | nessuno                     | Necessaria in produzione. |
| MTA_ALLOW_INSECURE_NO_AUTH | false                       | Solo sviluppo isolato.    |
| MTA_MAX_UPLOAD_MB          | 512                         | Limite upload.            |
| MTA_DATA_DIR               | /data/projects              | Storage persistente.      |
| XDG_CACHE_HOME             | /data/projects/.cache       | Cache modello/plugin.     |
| TORCH_HOME                 | /data/projects/.cache/torch | Cache Torch/Demucs.       |
| MTA_PORT                   | 8080                        | Porta HTTP.               |
| MTA_APP_VERSION            | VERSION                     | Versione applicativa.     |
| MTA_BUILD_ID               | BUILD                       | YYYYMMDD-HH:MM:SS.        |

## 5. Demucs/stem plugin

Il registry plugin espone stato e modelli supportati. L'adapter Demucs esegue il comando con argv fisso, senza shell. I modelli consentiti sono `htdemucs`, `htdemucs_ft` e `htdemucs_6s`. Il modello 6-stems è raccomandato quando sono richiesti guitar e piano separati.

Il primo utilizzo può richiedere accesso a Internet per ottenere i pesi. In cluster air-gapped predisporre la cache sul PVC prima dell'avvio operativo.

## 6. Pipeline audio e plugin

Gli insert sono mappati esclusivamente a preset FFmpeg presenti in un allow-list server-side. Il client non può inviare espressioni FFmpeg arbitrarie. Ogni traccia può avere fino a

16 insert EQ/normalizer/compressor/limiter. Il master applica una seconda catena separata sugli export stereo e sulle preview.

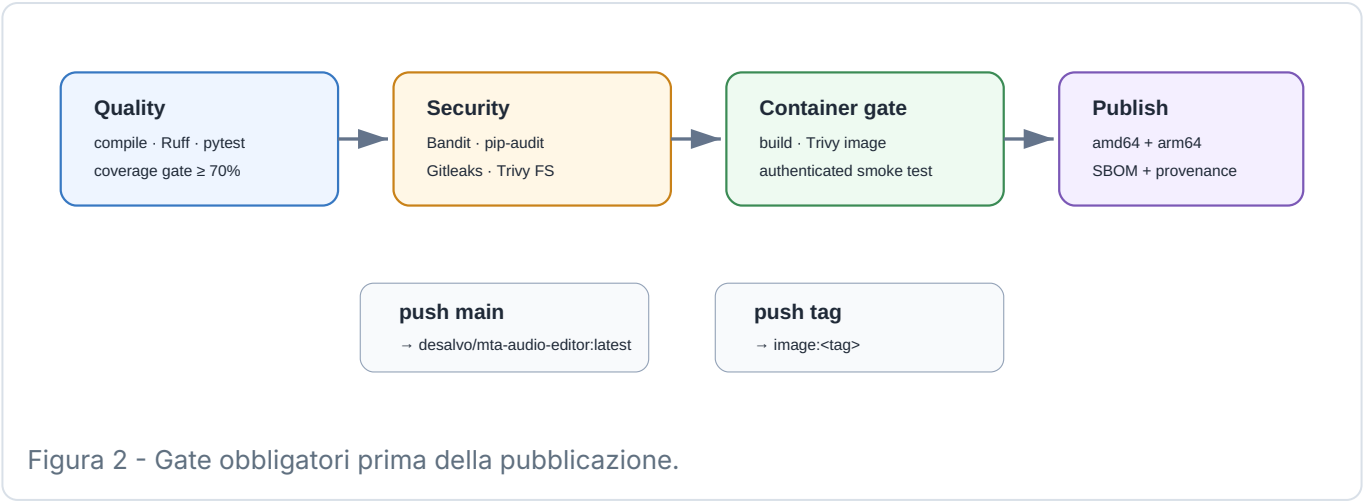
| Export               | Pipeline                                                                                                                                                    |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MTA                  | Clip → insert traccia → fader/pan → mapping slot/merge → stream separati → container Matroska/MTA. Il progetto non è limitato a 8/16 tracce durante il mix. |
| WAV/<br>MP3/<br>FLAC | Clip → insert traccia → fader/pan → summing → master fader → master inserts → encode. Le singole tracce sono esportabili separatamente in WAV/MP3/FLAC.     |

**Reverse analysis MTA v3:** all'import il sistema genera un report Matroska/attachment e decodifica le parti dimostrate dei file reali: frame testuali ID3v2.3, DELTA, wrapper LYRICS/COLORS/MIDITK/CHORDS e XML MtxInfoData. Il decoder recupera una keystream condivisa di 256 byte dalla sezione COLORS. LYRICS e CHORDS ricostruiscono timestamp minuto/centesimi e testo/accordi tramite XOR 0x30. COLORS usa record da 15 byte con quattro cifre tempo, tre cifre di posizione progressiva di highlighting, carry del minuto e record terminale distinto. MIDITK viene ricostruito byte-per-byte come Standard MIDI, salvato come sidecar `.miditk.mid` e ne vengono inventariati formato, PPQ, tempo e Marker meta-event. Il sidecar è esposto da `GET /api/projects/{pid}/mta-miditk`. La scrittura dei campi proprietari resta disabilitata finché non è validata sui dispositivi reali. Se FFmpeg non riesce a estrarre gli attachment, un fallback EBML FileData usa le dimensioni dichiarate da FFprobe per recuperarli in modo conservativo.

# 7. Sicurezza

| Controllo         | Implementazione                                            |
|-------------------|------------------------------------------------------------|
| Auth              | HTTP Basic di default, nessuna password incorporata.       |
| Request integrity | X-MTA-Request : 1 sulle richieste mutative.                |
| Path traversal    | ID e filename confinati nelle directory progetto.          |
| Upload            | Limite, estensione e FFprobe.                              |
| Plugin processing | Preset allow-listed; niente filtri arbitrary dal browser.  |
| Process execution | FFmpeg, FFprobe e Demucs senza shell.                      |
| HTTP headers      | CSP, nosniff, frame deny, no-referrer, Permissions-Policy. |
| Container         | Non-root, cap-drop ALL, seccomp e no privilege escalation. |

# 8. CI/CD



Quality, test/coverage, Ruff, Bandit, pip-audit, Gitleaks, Trivy filesystem/image, container smoke test e CodeQL. CI e immagine production usano Python 3.14. Un push su main pubblica desalvo/mta-audio-editor:latest; un tag pubblica lo stesso tag. Le immagini sono amd64/arm64 con SBOM e provenance.

## 9. Operazioni

---

- Terminare TLS su Ingress/reverse proxy.
- Monitorare spazio PVC: sorgenti audio, export e cache modello possono occupare molto spazio.
- Dimensionare CPU/RAM in base alla separazione stems; Demucs è sensibilmente più pesante del normale editing.
- Centralizzare i log e monitorare probe/restart.
- Mantenere aggiornati base image e dipendenze tramite Dependabot e gate.

## 10. Backup/restore

---

Eseguire snapshot/backup coerenti del PVC. Il dato essenziale è /data/projects. La sottodirectory .cache può essere esclusa per ridurre il backup, sapendo che i modelli dovranno essere riscaricati o ripristinati separatamente.

## 11. Release e versioning

---

Release corrente: **0.2.0-11**. Ogni package aggiorna automaticamente BUILD nel formato YYYYMMDD-HH:MM:SS. Esempio:

```
./scripts/package.sh
git add .
git commit -m "Release 0.2.0-11"
git push origin main
git tag -s 0.2.0-11 -m "MTA Audio Editor 0.2.0-11"
git push origin 0.2.0-11
```

## 12. Responsabilità production

---

La baseline include hardening, test e gate automatizzati. Restano responsabilità dell'operatore: TLS, segreti, backup, monitoraggio, network policy, capacity planning, patch management e validazione funzionale sui dispositivi MTA reali. La compatibilità bit-perfect con metadati proprietari Merish non documentati non è dichiarata.

## 12.1 Reverse engineering MTA controllato

| Artefatto                         | Analisi                                                                                                                                                                                                                               | Politica di scrittura                                                                         |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Container                         | FFprobe/Matroska: stream, codec, tag, attachment.                                                                                                                                                                                     | Ricostruzione controllata in export.                                                          |
| SYL / attachment proprietario     | ID3v2.3 completo, frame testuali, DELTA e fingerprint dei payload.                                                                                                                                                                    | Preservato byte-per-byte finché il campo non è documentato o validato.                        |
| MtxInfoData XML                   | Identity, General, tracce, routing/volume/mute e statistiche NoteOn; padding terminale rilevato.                                                                                                                                      | Lettura strutturata; nessuna inferenza nascosta sulla soglia NoteOn.                          |
| LYRICS / CHORDS / COLORS / MIDITK | Decodifica read-only validata su quattro file stock: LYRICS/CHORDS con testo/accordi e timing minuto/centesimi; COLORS con record da 15 byte, timing e posizione progressiva; MIDITK ricostruito come Standard MIDI con tempo/marker. | Nessuna mutazione speculativa; i sentinel e i campi non ancora dimostrati restano preservati. |

Per chiudere la compatibilità di scrittura creare coppie di MTA dello stesso brano variando un solo elemento alla volta (una parola, un accordo, un marker, un colore). L'endpoint `/api/mta/diff` restituisce prefisso/suffisso comune e range binari modificati.

## 13. Plugin custom e Auto Mix

### 13.1 Registry e preset

I processori sono definiti nel registry server-side `app/plugins.py`. Il browser non invia espressioni FFmpeg arbitrarie: invia solo tipo plugin, preset o parametri numerici validati rispetto a uno schema con range min/max. I preset custom sono persistiti in `/data/projects/_custom_presets.json`; includere questo file nei backup del PVC.

| Controllo          | Implementazione                                                                                          |
|--------------------|----------------------------------------------------------------------------------------------------------|
| Factory preset     | Immutabili, definiti nel codice.                                                                         |
| User preset        | Parametri JSON validati e salvati sul volume persistente.                                                |
| Raw DSP expression | Non accettata dalle API.                                                                                 |
| Catena             | Massimo 16 insert per traccia/master, applicati in ordine.                                               |
| Lexicon-style      | Preset ispirati a plate/hall/ambient usando filtri FFmpeg aperti; nessun algoritmo proprietario Lexicon. |

## 13.2 Auto Mix

L'endpoint `POST /api/projects/{pid}/auto-mix` riceve `{enabled, style}`. All'attivazione salva uno snapshot completo di volume, pan e insert di ogni traccia più volume/insert master. Le regole sono deterministiche e type-aware; alla disattivazione lo snapshot viene ripristinato. Questo evita drift cumulativo e rende il wizard idempotente dal punto di vista operativo.

## 13.3 Preset API

```
POST /api/presets
{
  "plugin": "amplify",
  "name": "My Trim",
  "params": {"gain_db": -3.0}
}

DELETE /api/presets/amplify/My%20Trim
```